PREDICTIVE ANALYTICS — CA3 PROJECT REPORT

E-Commerce Product Return Prediction

Using Machine Learning — an end-to-end KDD study

Student	Bhargav Sor
PRN	23070521031
Programme	B.Tech, Computer Science and Engineering
Semester / Section	VII / A
Institute	Symbiosis Institute of Technology, Nagpur
Course	Predictive Analytics (CA3)
Dataset	Kaggle — E-Commerce Dataset (hakdevelopment/e-commerce-dataset): train.csv, test.csv, sample_submission.csv
Conference link	ICSIT 2026 (2nd International Conference on Sustainability, Innovation and Technology), SIT Nagpur

How to read this report: green boxes contain results actually produced by running the Python pipeline on the dataset; amber boxes describe methodology, design choices or proposals. No number in this report was typed by hand — all values are read from the pipeline's results file.

Contents

(Right-click the table above and choose “Update Field” in Word to refresh page numbers.)

Executive Summary

OBSERVED RESULT (from the executed code)

The dataset **does contain a direct target**: the column `returned` (0 = kept, 1 = returned) in `train.csv`. `test.csv` is unlabelled (Kaggle-style), so all evaluation uses a stratified split of the 200,000 labelled orders.

The classes are nearly balanced: 47.5% of orders were returned. There are no missing values and no duplicate rows, but 32.0% of rows contain at least one physically impossible value (e.g., negative prices or view counts, ratings outside 1–5), which suggests synthetic or noise-injected data.

Every feature is only weakly related to returns (largest absolute point-biserial correlation 0.055).

The best model by 5-fold cross-validated ROC-AUC was **XGBoost** (CV ROC-AUC 0.5947 ± 0.0038; holdout 0.5957, bootstrap 95% CI 0.590 – 0.601). Logistic Regression was close behind (holdout 0.5908).

At the 0.5 threshold the best model reached accuracy 0.571, precision 0.562, recall 0.438 and F1 0.492. Ranking is more useful than a hard label: orders in the highest-risk 10% were returned 63.1% of the time vs 33.6% in the lowest-risk 10% (base rate 47.5%).

Conclusion: the dataset technically supports the project, but its features carry little predictive signal. The model is better than chance but only modestly, so it is suitable as a risk-ranking aid and a methodological demonstration, not as a reliable per-order return predictor.

1. Problem Definition

Product returns are a major cost in e-commerce: each return involves reverse logistics, inspection, restocking or disposal, and often a refund of shipping costs. If the probability of a return can be estimated early, a retailer can act before the cost is incurred — for example, by showing better size or product information, confirming high-risk orders, adjusting discount or coupon policies, or planning reverse-logistics capacity.

1.1 Objective

To build and evaluate a binary classification model that predicts whether an e-commerce order will be returned (`returned` = 1) or kept (`returned` = 0), using only information that is available before the return happens, and to interpret which factors are associated with returns.

1.2 Prediction moment and success criteria

METHODOLOGY / PROPOSED

Prediction moment. Two moments are considered: (a) **at checkout**, using only order, customer-history and browsing features; and (b) **after delivery**, additionally using the delivery delay. Both are before the customer decides to return, so neither leaks the outcome.

Success criteria. The model must (1) beat the trivial baselines on ROC-AUC and F1, (2) show a statistically reliable improvement (bootstrap confidence intervals), and (3) produce explanations that a business user can act on.

Conference link. Dr. Nidhi Rohatgi (ICSIT 2026) stressed that a prediction without an action plan has little value, and Dr. Siddharth Jabade stressed confirming that the problem is a real, industry-relevant problem before solving it. The project is therefore framed around a decision (which orders to act on), not just a label.

2. Data Selection and Understanding

2.1 Files

File	Rows	Columns	Contents
train.csv	200,000	16	Labelled orders: 14 predictors + target `returned` + identifier `order_id`
test.csv	50,000	15	Same predictors + `order_id`; **no target** (target present: no)
sample_submission.csv	50,000	2	Columns: ID, returned — IDs match test.csv order: yes

OBSERVED RESULT (from the executed code)

Target found: `returned` is a genuine binary Return / No-Return label (values 0 and 1). It is not derived or invented. The sample submission's column header (`returned`) confirms it is the intended prediction target.

Data quality: missing values — 0 in train, 0 in test; duplicate rows — 0 (and 0 ignoring `order_id`); duplicate order IDs — 0; order IDs shared between train and test — 0.

Class balance: 105,081 kept (52.5%) vs 94,919 returned (47.5%). The classes are close to balanced, so resampling (e.g., SMOTE) is not needed.

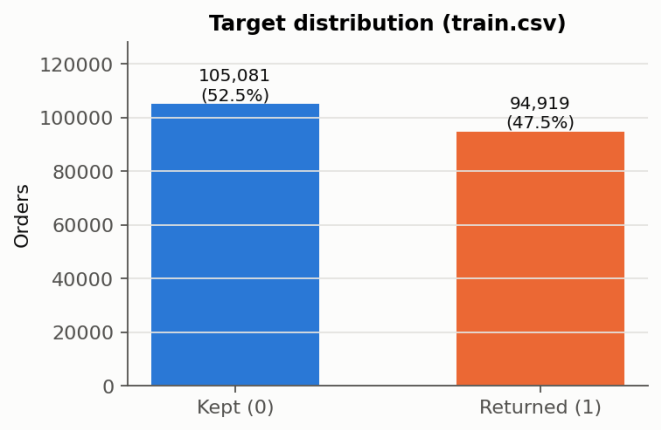


Figure 1. Distribution of the target variable in train.csv.

2.2 Data dictionary

The table lists every column exactly as it appears in the files, with the observed range and when the information would be known in a real business process. The descriptions are inferred from the column names, because the dataset page provides no official documentation; this is stated as a limitation.

Column	Type	Observed values	When known / role
customer_age	Numeric (int)	10.00 to 87.00	Customer profile - known before/at checkout
product_price	Numeric	-183.29 to 855.08	Order line - known at checkout
discount_percent	Numeric	-7.54 to 78.79	Order line - known at checkout
product_rating	Numeric	0.45 to 5.45	Product listing - assumed to be the product's average rating shown before purchase (dataset does not document it)

Column	Type	Observed values	When known / role
past_purchase_count	Numeric (int)	1.00 to 26.00	Customer history - known at checkout
past_return_rate	Numeric	-0.06 to 0.87	Customer history - known at checkout IF computed from earlier orders only (dataset does not document it)
delivery_delay_days	Numeric	-9.05 to 7.96	Logistics - known only after delivery (still before a return decision)
session_length_minutes	Numeric	-11.01 to 137.65	Browsing session - known at checkout
num_product_views	Numeric (int)	-5.00 to 44.00	Browsing session - known at checkout
device_type	Categorical	mobile, desktop, tablet	Browsing session - known at checkout
product_category	Categorical	toys, beauty, electronics, home, clothing, sports	Order line - known at checkout
shipping_method	Categorical	standard, express, same_day	Order line - known at checkout
payment_method	Categorical	debit_card, credit_card, apple_pay, paypal	Order line - known at checkout
used_coupon	Binary (0/1)	0, 1	Order line - known at checkout
order_id	Identifier	0 to 249,999 (unique)	Identifier - excluded (no predictive meaning)
returned	Binary target	0 = kept, 1 = returned	TARGET - known only after the return window

Variable groups. Numerical (9): customer_age, product_price, discount_percent, product_rating, past_purchase_count, past_return_rate, delivery_delay_days, session_length_minutes, num_product_views. Categorical (4): device_type, product_category, shipping_method, payment_method. Binary (1): used_coupon. Identifier (1): order_id. Target (1): returned.

2.3 Physically impossible values

Column	Valid range	Rows outside range	% of rows	Return rate in those rows
product_price	0 to inf	27,285	13.64%	43.8%
discount_percent	0 to 100	1,686	0.84%	41.5%
product_rating	1 to 5	18,111	9.06%	46.3%
past_return_rate	0 to 1	2,220	1.11%	42.2%
session_length_minutes	0 to inf	1,487	0.74%	51.5%
num_product_views	0 to inf	21,443	10.72%	50.7%
customer_age < 18 (flag only, kept)	flag	2,109	1.05%	49.4%

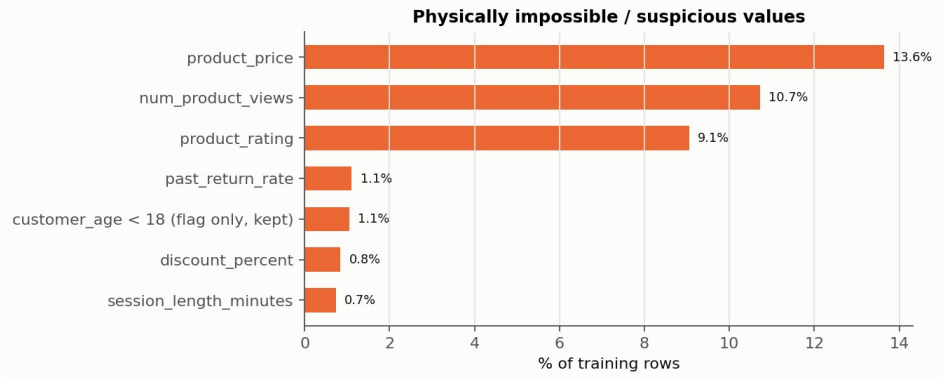


Figure 2. Share of training rows with impossible or suspicious values.

OBSERVED RESULT (from the executed code)

64,079 rows (32.0%) have at least one impossible value. Values such as a negative number of product views or a negative price cannot come from a real transactional system. Together with the fact that almost every numeric column has nearly as many unique values as rows (continuous noise even in fields such as rating), this strongly suggests that the dataset is **synthetic or noise-injected**. This is an observation about the data, not a verified fact about its origin.

3. Data Pre-processing

3.1 Data-cleaning plan

METHODOLOGY / PROPOSED

1. Missing values and duplicates — checked; none found, so no rows are removed.

2. Identifier — `order_id` is dropped from the features. Its single-feature ROC-AUC was also checked to confirm that it does not leak the target.

3. Impossible values — values outside the valid range of each field (Section 2.3) are set to missing, then imputed with the training-fold median, and a **missing-indicator** column is added so the model can still learn whether a value was invalid.

4. Age below 18 — kept but noted, since young buyers are unusual but not impossible.

5. Leakage protection — every cleaning, imputation, encoding and scaling step is inside a scikit-learn `Pipeline`, so its parameters are learned only from the training folds.

3.2 Does cleaning matter? (ablation)

OBSERVED RESULT (from the executed code)

Using the best model (XGBoost) and 5-fold CV: **impossible values -> NaN + median impute + indicator** → ROC-AUC 0.5947 ± 0.0038; **raw values (no cleaning)** → ROC-AUC 0.5953 ± 0.0032.

The difference (-0.0005) is smaller than one standard deviation, so cleaning neither helps nor hurts predictive performance here. It is kept because it makes the data semantically valid and the model's explanations interpretable.

4. Data Transformation and Feature Engineering

4.1 Engineered features

Feature	Formula	Rationale
final_price	product_price × (1 − discount_percent/100)	What the customer actually paid; price paid may affect return behaviour
discount_amount	product_price × discount_percent/100	Absolute saving; deep discounts can encourage impulse buying
expected_prior_returns	past_purchase_count × past_return_rate	Approximate number of earlier returns by this customer
views_per_minute	num_product_views / max(session_length_minutes, 1)	Browsing intensity: comparison-shopping vs quick purchase
is_late (post-delivery only)	1 if delivery_delay_days > 0	Late delivery as a possible return trigger
missing-indicator columns	1 if a value was invalid and imputed	Keeps information about invalid values after cleaning

METHODOLOGY / PROPOSED

Encoding and scaling. The 4 categorical columns are one-hot encoded (with the first level dropped for Logistic Regression, to avoid perfect collinearity). Numerical features are standardised for Logistic Regression only; tree models do not need scaling.

All engineered features use row-level arithmetic only, so they cannot leak information between orders or from the test set.

4.2 Leakage review: what is known before a return?

Feature	Available at checkout?	Available after delivery (before return)?	Decision
Order, product, session, payment, coupon features	Yes	Yes	Used
customer_age, past_purchase_count	Yes	Yes	Used
past_return_rate	Yes — only if computed from earlier orders	Yes	Used; flagged as an undocumented assumption
product_rating	Yes — if it is the product's average listing rating	Yes	Used; flagged (a post-purchase review rating would be leakage)
delivery_delay_days	No	Yes	Used only in the post-delivery scenario
order_id	—	—	Excluded (identifier)
returned	No	No	Target only

OBSERVED RESULT (from the executed code)

No leakage signal found. No single feature predicts the target strongly on its own (every single-feature ROC-AUC is close to 0.5), and `order_id` alone gives ROC-AUC 0.4988. A leaked post-return feature would normally show a much higher AUC.

Scenario comparison (XGBoost, 5-fold CV): checkout (excludes delivery_delay_days) → ROC-AUC 0.5932 ± 0.0040; post-delivery (includes delivery_delay_days) → ROC-AUC 0.5947 ± 0.0038. Adding delivery delay changes ROC-AUC by only 0.0016, so a checkout-time model loses almost nothing and can act earlier. The main results below use the post-delivery feature set.

5. Exploratory Data Analysis

5.1 EDA plan

METHODOLOGY / PROPOSED

(1) Target balance; (2) distribution of each numeric feature split by class; (3) return rate for each categorical level; (4) correlation matrix; (5) formal hypothesis tests — Mann–Whitney U for numeric features (the distributions are skewed or noisy, so a non-parametric test is used) and Chi-square for categorical features, at α = 0.05; (6) **effect sizes** (point-biserial r, Cramér's V), because with 200,000 rows even tiny, practically irrelevant differences are statistically significant.

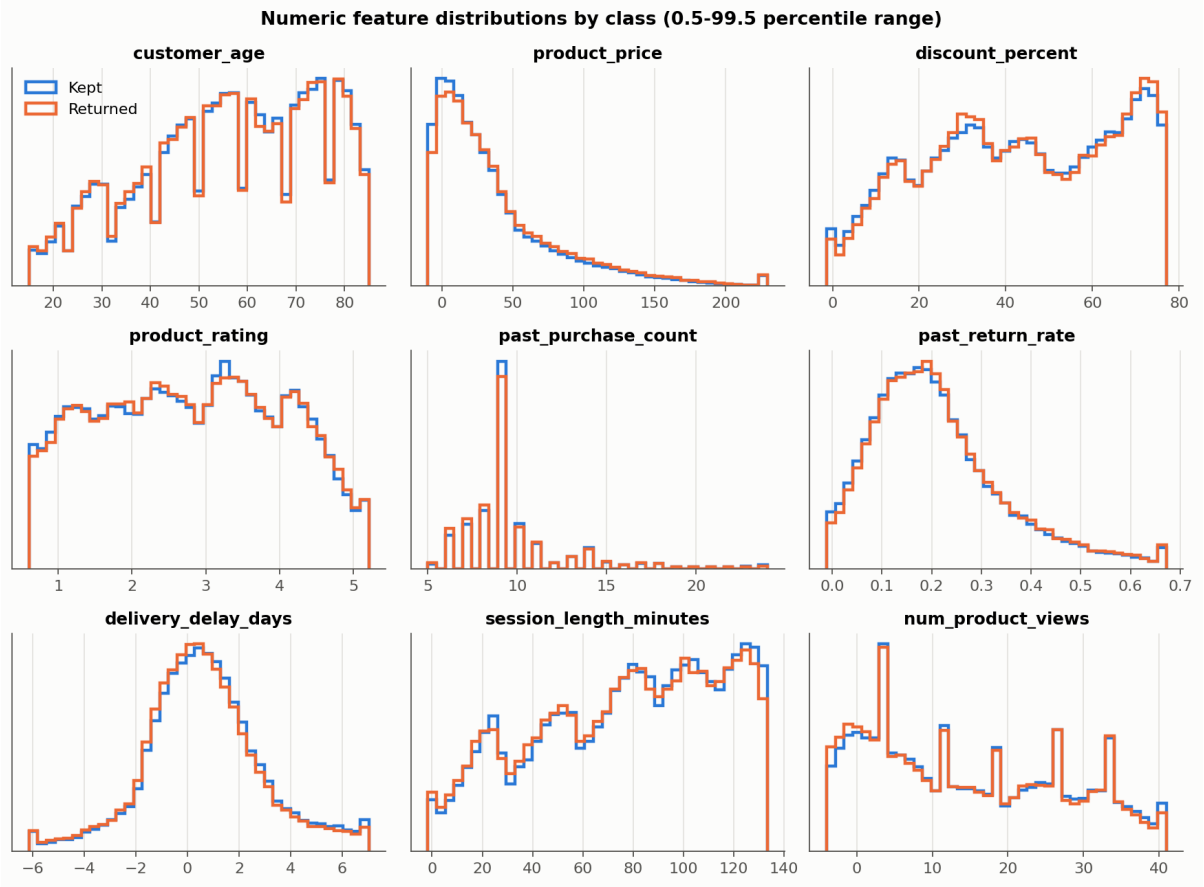


Figure 3. Numeric feature distributions for kept (blue) and returned (orange) orders.

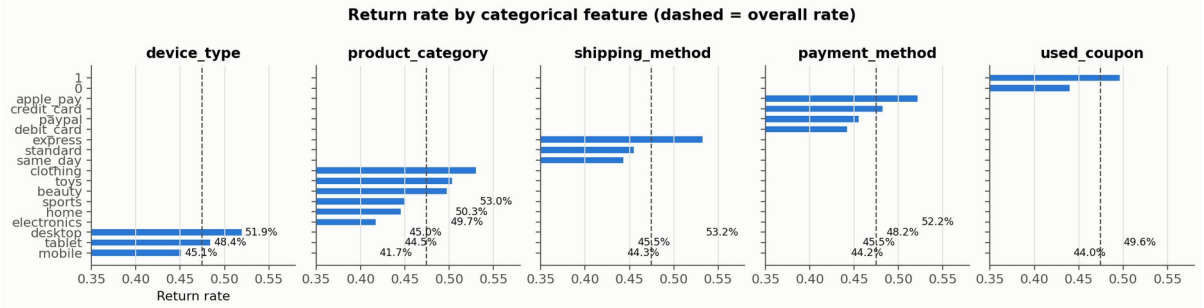


Figure 4. Return rate by categorical feature level (dashed line = overall rate).

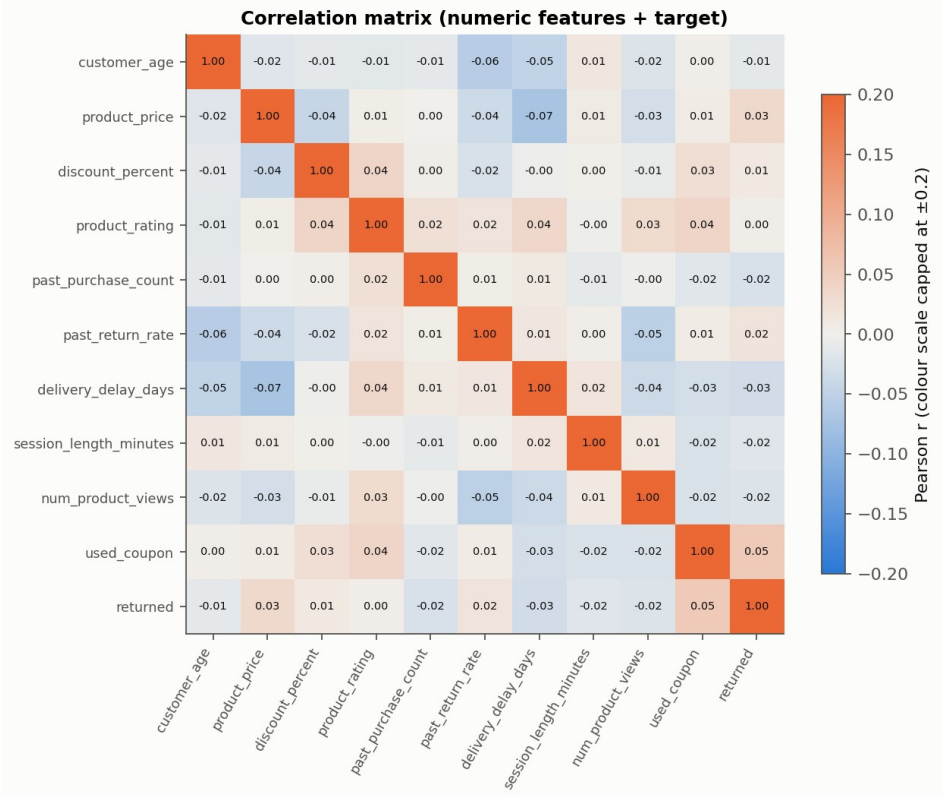


Figure 5. Correlation matrix of numeric features and the target (colour scale capped at ±0.2).

5.2 Hypothesis tests

ID	Hypothesis	Mean (returned vs kept)	Effect size r	Mann–Whitney p	Outcome
H1	Returned orders have higher prices (`product_price`)	39.632 vs 36.496	0.034	1.7×10^-71	Supported (tiny effect)
H2	Coupon use is associated with returns (`used_coupon`)	0.644 vs 0.590	0.055	2.4×10^-133	Supported (tiny effect)
H3	Longer delivery delays lead to more returns (`delivery_delay_days`)	0.548 vs 0.695	-0.032	5.1×10^-48	Significant but **opposite direction**
H4	Customers with higher past return rates return more (`past_return_rate`)	0.218 vs 0.213	0.018	1.0×10^-16	Supported (tiny effect)
H5	Lower-rated products are returned more (`product_rating`)	2.819 vs 2.808	0.005	0.0581	Not significant
H6	Frequent buyers return less (`past_purchase_count`)	9.542 vs 9.691	-0.024	1.7×10^-47	Supported (tiny effect)

ID	Categorical feature	Highest / lowest return rate	Cramér's V	Chi-square p	Outcome
H7	`product_category` is associated with returns	clothing 53.0% / electronics 41.7%	0.076	1.4×10^-250	Supported (weak association)
H8	`shipping_method` is associated with returns	express 53.2% / same_day 44.3%	0.070	3.6×10^-211	Supported (weak association)
H9	`device_type` is associated with returns	desktop 51.9% / mobile 45.1%	0.057	2.1×10^-140	Supported (weak association)

ID	Categorical feature	Highest / lowest return rate	Cramér's V	Chi-square p	Outcome
H10	`payment_method` is associated with returns	apple_pay 52.2% / debit_card 44.2%	0.061	5.5×10^-160	Supported (weak association)

OBSERVED RESULT (from the executed code)

Most relationships are statistically significant only because the sample is very large; every effect size is **very small** (|r| ≤ 0.055, Cramér's V ≤ 0.076).

Two findings go against common business expectations: returned orders had **lower** average delivery delay (0.548 vs 0.695 days), and product rating showed no significant difference (p = 0.0581). These are reported as observed; they may reflect how the (likely synthetic) data was generated rather than real customer behaviour.

The categorical variables show the clearest differences: clothing has the highest return rate (53.0%) and electronics the lowest (41.7%); express shipping is highest (53.2%).

6. Model Development

METHODOLOGY / PROPOSED

Split. train.csv → stratified 80% training part (160,000 orders) and 20% holdout (40,000 orders, return rate 47.5%). The holdout is used only once, for the final evaluation. test.csv cannot be used for evaluation because it has no labels.

Models. Two trivial baselines (always predict "kept"; always predict "returned"); **Logistic Regression** (interpretable baseline); **Decision Tree**; **Random Forest**; **Histogram Gradient Boosting** (scikit-learn); and **XGBoost**. Other algorithms (e.g., SVM, KNN) were not added: they scale poorly to 160,000 rows and are not expected to capture different structure than the models above.

Tuning. A grid search with 3-fold stratified CV on a 50,000-order stratified subsample of the training part (to fit the compute budget), optimising ROC-AUC. The selected settings were then evaluated with 5-fold stratified CV on the full training part.

Model	Selected hyper-parameters	Tuning CV ROC-AUC
Logistic Regression	C = 0.1	0.5886
Decision Tree	max_depth = 6, min_samples_leaf = 200	0.5719
Random Forest	max_depth = 8, min_samples_leaf = 50	0.5873
Hist Gradient Boosting	l2_regularization = 1, learning_rate = 0.05, max_leaf_nodes = 15	0.5871
XGBoost	learning_rate = 0.05, max_depth = 3	0.5889

7. Model Evaluation

7.1 Evaluation strategy and why each metric is used

Metric	What it measures	Why it matters for return prediction
Accuracy	Share of all orders classified correctly	Easy to read, but only meaningful against the 52.5% majority-class baseline
Precision	Of orders flagged as returns, the share that were returned	Cost of acting on a false alarm (e.g., an unnecessary confirmation call or withdrawn coupon)
Recall	Of actual returns, the share that were flagged	How many costly returns the retailer gets a chance to prevent
F1-score	Harmonic mean of precision and recall	A single balance of the two; compared against the "always returned" baseline
ROC-AUC	Ranking quality across all thresholds	Threshold-independent; 0.5 means no better than random ordering
PR-AUC	Average precision across recall levels	Shows gain over the base rate for the positive (returned) class
Confusion matrix	Counts of TN, FP, FN, TP	Makes the business trade-off concrete

METHODOLOGY / PROPOSED

Class balance check. The return rate is 47.5%, so the classes are close to balanced. Stratified splits are used, no resampling is needed, and accuracy is still reported, but always next to the trivial baselines.

Reliability. Metrics are reported as the 5-fold CV mean ± standard deviation, then confirmed once on the untouched holdout. **Bootstrap** resampling of the holdout (1000 resamples) gives 95% confidence intervals and tests whether the best model really beats Logistic Regression.

7.2 Cross-validated model comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC (± SD)	PR-AUC
Baseline (majority class)	0.5254	0.0000	0.0000	0.0000	0.5000 ± 0.0000	0.4746
Logistic Regression	0.5662	0.5550	0.4331	0.4865	0.5895 ± 0.0039	0.5554
Decision Tree	0.5587	0.5535	0.3641	0.4390	0.5774 ± 0.0029	0.5411
Random Forest	0.5657	0.5675	0.3574	0.4385	0.5909 ± 0.0038	0.5579
Hist Gradient Boosting	0.5691	0.5594	0.4335	0.4884	0.5936 ± 0.0033	0.5611
XGBoost	**0.5701**	**0.5604**	**0.4371**	**0.4911**	**0.5947 ± 0.0038**	**0.5619**

Table: 5-fold stratified CV on the 160,000-order training part (threshold 0.5). Highlighted row = best by ROC-AUC.

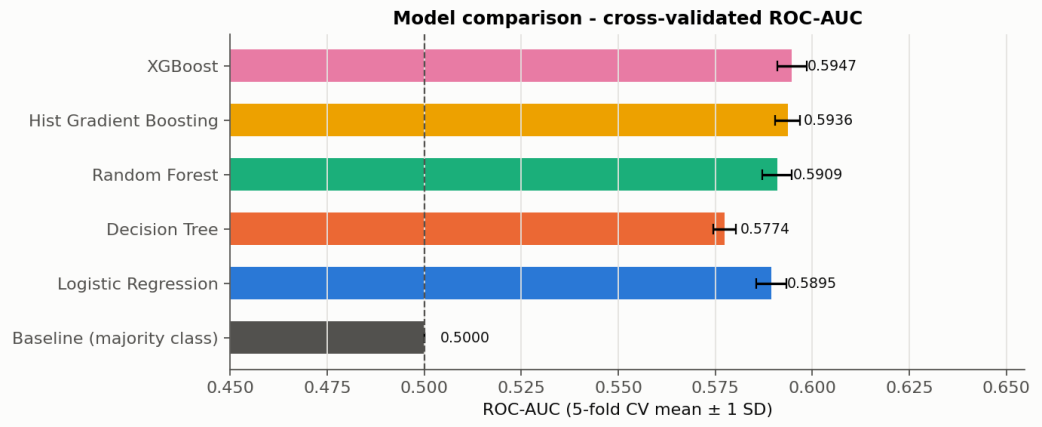


Figure 6. Cross-validated ROC-AUC by model (error bars = ±1 SD across folds).

7.3 Holdout results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC
Baseline (majority class)	0.5254	0.0000	0.0000	0.0000	0.5000	0.4746
Baseline (always predict returned)	0.4746	0.4746	1.0000	0.6437	0.5000	0.4746
Logistic Regression	0.5672	0.5558	0.4381	0.4900	0.5908	0.5572
Decision Tree	0.5555	0.5473	0.3668	0.4392	0.5756	0.5386
Random Forest	0.5697	0.5728	0.3673	0.4476	0.5932	0.5597
Hist Gradient Boosting	0.5698	0.5596	0.4391	0.4921	0.5943	0.5610
XGBoost	**0.5715**	**0.5623**	**0.4379**	**0.4924**	**0.5957**	**0.5620**

Table: results on the 40,000-order holdout at threshold 0.5.

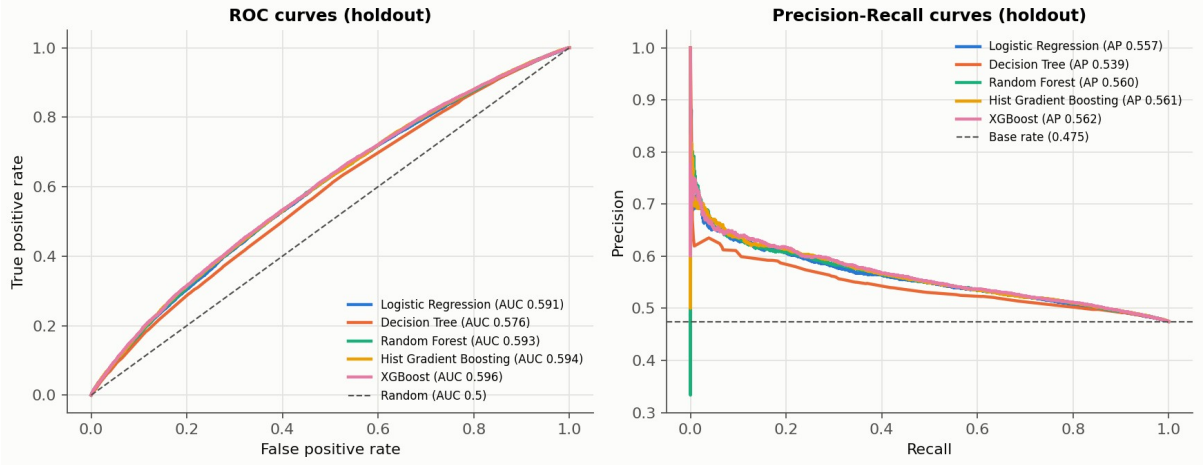


Figure 7. ROC and precision–recall curves on the holdout.

OBSERVED RESULT (from the executed code)

XGBoost had the best ranking quality: holdout ROC-AUC 0.5957 (95% CI 0.590 – 0.601) vs Logistic Regression 0.5908 (95% CI 0.585 – 0.596).

The bootstrap 95% CI of the AUC difference (XGBoost − LR) is 0.003 – 0.007. It excludes zero, so the gain is statistically real, but it is small in practical terms: the interpretable Logistic Regression captures most of the available signal.

All models beat the majority-class baseline on ROC-AUC and F1. The “always predict returned” rule has F1 0.644, which is **higher** than every model's F1 at the 0.5 threshold (best 0.492). F1 therefore rewards flagging everything, and should not be used alone to judge these models.

7.4 Threshold choice

OBSERVED RESULT (from the executed code)

Choosing the threshold that maximises F1 on out-of-fold predictions gave 0.341, which flags 95.5% of orders as returns. Its F1 (0.645) is essentially the same as flagging every order (0.644). This threshold is **rejected** as degenerate. On the holdout it gives recall 0.971 but precision only 0.482.

The model is therefore used at the default 0.5 threshold, or — more usefully — as a **ranking** tool that flags a fixed share of the highest-risk orders for action.

Threshold	Share flagged	Precision	Recall	F1	Accuracy
0.40	82.2%	0.500	0.867	0.635	0.526
0.45	61.5%	0.527	0.682	0.594	0.558
0.50	37.0%	0.562	0.438	0.492	0.571
0.55	17.5%	0.606	0.223	0.326	0.563
0.60	6.5%	0.645	0.088	0.155	0.544

Table: precision–recall trade-off for XGBoost on the holdout.

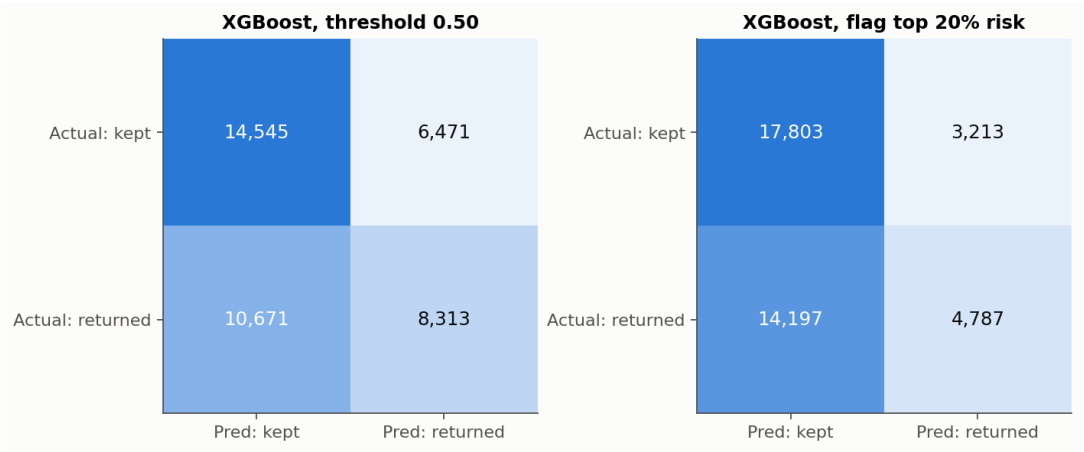


Figure 8. Confusion matrices for XGBoost: threshold 0.5 (left) and flagging the top 20% highest-risk orders (right).

At threshold 0.5: TN = 14,545, FP = 6,471, FN = 10,671, TP = 8,313. When only the top 20% highest-risk orders are flagged: TN = 17,803, FP = 3,213, FN = 14,197, TP = 4,787, i.e. precision 59.8% for the flagged group.

7.5 Lift by risk decile

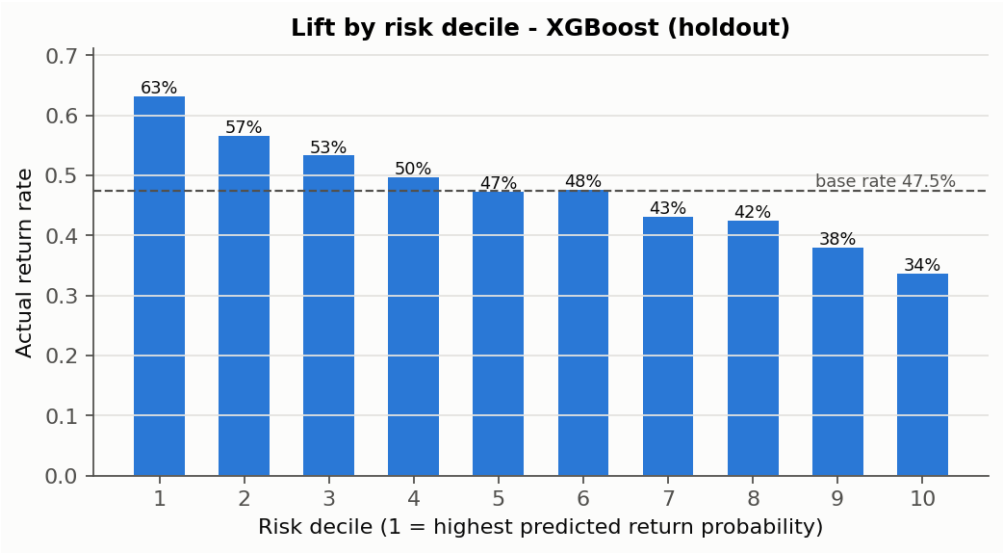


Figure 9. Actual return rate in each decile of predicted risk (XGBoost, holdout).

OBSERVED RESULT (from the executed code)

Orders in the highest-risk decile were returned 63.1% of the time (lift 1.33×; bootstrap 95% CI 61.7% – 64.8%), vs 33.6% in the lowest-risk decile.

This is the most practical result: the model can rank orders so that interventions are focused where returns are most likely. It cannot reliably identify individual returns.

8. Interpretation and Explainability

8.1 Explainability approach

METHODOLOGY / PROPOSED

Global, model-specific: Logistic Regression odds ratios (effect per 1 standard deviation for numeric features; vs the reference level for categories).

Global, model-agnostic: permutation importance for XGBoost — the drop in holdout ROC-AUC when a feature's values are shuffled.

Local and global, tree-specific: SHAP values for XGBoost on 3,000 holdout orders, showing the direction and size of each feature's contribution to individual predictions.

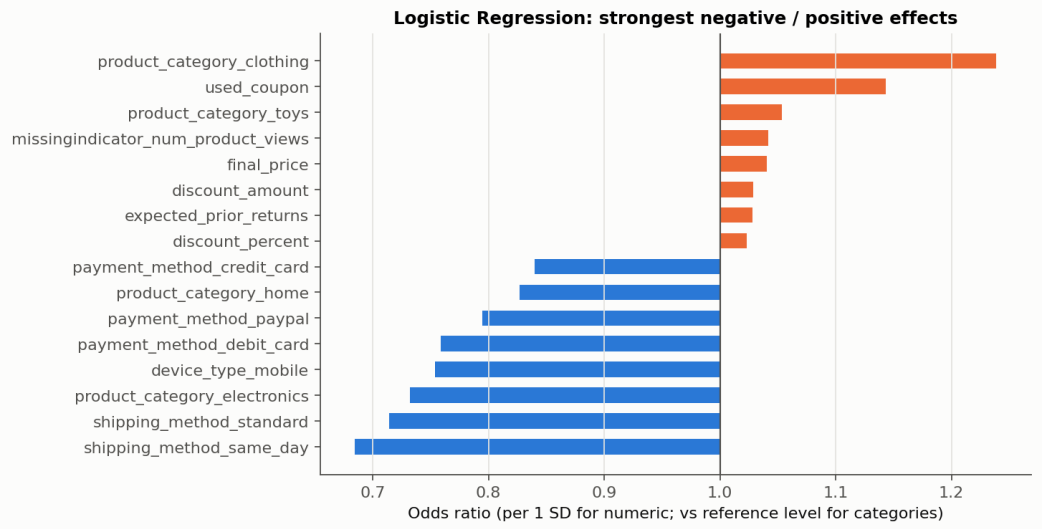


Figure 10. Logistic Regression odds ratios (orange > 1 increases return odds; blue < 1 decreases them).

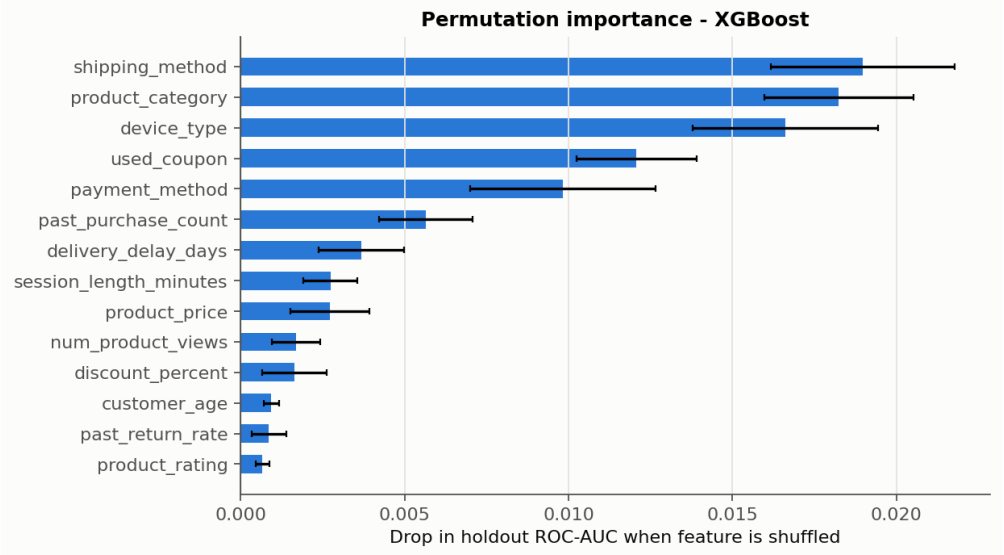


Figure 11. Permutation importance for XGBoost (holdout).

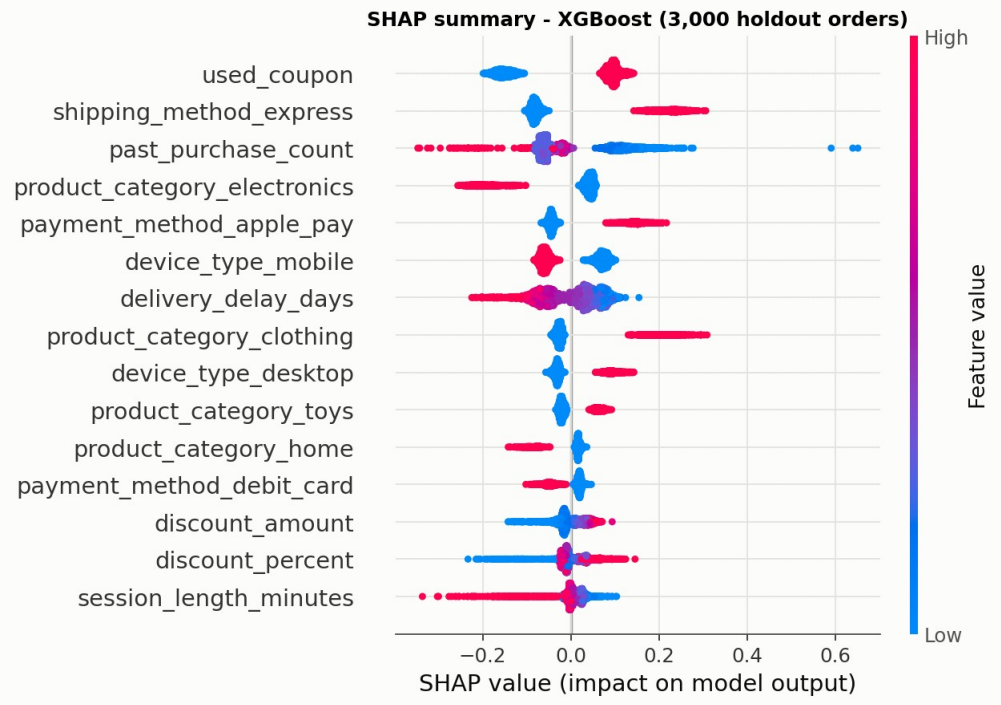


Figure 12. SHAP summary for XGBoost (3,000 holdout orders).

8.2 What drives returns in this dataset?

OBSERVED RESULT (from the executed code)

Permutation importance (top 5): shipping_method (0.0190), product_category (0.0182), device_type (0.0166), used_coupon (0.0121), payment_method (0.0098). The categorical order attributes contribute the most; customer age, past return rate and product rating contribute the least.

Logistic Regression — largest increases in return odds: product_category_clothing (OR 1.239), used_coupon (OR 1.143), product_category_toys (OR 1.053), missingindicator_num_product_views (OR 1.041).

Largest decreases: shipping_method_same_day (OR 0.685), shipping_method_standard (OR 0.714), product_category_electronics (OR 0.732), device_type_mobile (OR 0.754).

All effects are modest; the explanations describe weak tendencies, not strong rules. Features one would expect to matter a lot, such as past return rate and product rating, contribute very little in this data.

8.3 Business interpretation

METHODOLOGY / PROPOSED

Use the model as a risk ranker. For example, flag the top 10–20% highest-risk orders for low-cost interventions: clearer size or product information, an order-confirmation prompt, or reviewing coupon eligibility.

Category and shipping policy. The categories and shipping methods with the highest return rates are candidates for better product content and review of delivery promises.

Coupons. Coupon use is associated with more returns, so promotion design could be reviewed (e.g., whether coupons attract low-intent purchases).

Do not use the model to penalise individual customers: its per-order accuracy is too low for decisions with significant consequences for the customer.

9. KDD Process Mapping

KDD stage	What was done	Evidence in this report
1. Problem definition	Binary return prediction framed around an action (which orders to act on); two prediction moments	Section 1
2. Data selection & understanding	Inspected all 3 files; confirmed a genuine target; data dictionary; class balance; quality audit	Section 2, Fig. 1–2
3. Pre-processing	Impossible values → NaN + median imputation + indicators; ID removed; leakage-safe pipeline; ablation	Section 3
4. Transformation / feature engineering	5 engineered features; one-hot encoding; scaling for LR; availability-based leakage review	Section 4
5. Exploratory data analysis	Class-wise distributions, category return rates, correlations, 10 hypothesis tests with effect sizes	Section 5, Fig. 3–5
6. Model development	2 baselines + 5 classifiers; grid-search tuning; 5-fold stratified CV	Section 6
7. Model evaluation	Holdout metrics, ROC/PR curves, confusion matrices, threshold analysis, lift, bootstrap CIs	Section 7, Fig. 6–9
8. Interpretation	Odds ratios, permutation importance, SHAP, business actions, limitations	Sections 8, 11

10. Connection to ICSIT 2026 Conference Sessions

Only ideas that genuinely apply to this project are used. The mapping below is based on the conference transcripts.

10.1 Dr. Pastor R. Arguelles Jr. — Impact of Big Data on Supply Chain Performance Through Demand Forecasting

- **Applies:** he framed data analytics as moving from descriptive to predictive to prescriptive, with results presented as **actionable insights**. This project delivers a risk ranking linked to concrete actions (Section 8.3), and return prediction also supports reverse-logistics planning, which is part of supply-chain performance.
- **Applies:** he checked reliability before interpreting results and tested hypotheses using bootstrapping. Here, formal hypothesis tests with effect sizes (Section 5.2) and bootstrap confidence intervals (Section 7) play the same role.
- **Applies:** his emphasis on data privacy and cyber-security, relevant because return prediction uses customer-level history.
- **Not used:** SmartPLS, construct-reliability measures (Cronbach's Alpha, AVE, HTMT), IoT and blockchain. These belong to survey-based structural equation modelling and supply-chain infrastructure; this project uses observational tabular data and classification models, so they do not apply.

10.2 Dr. Nidhi Rohatgi — Engineering AI for Real Clinical Impact

- **Applies:** a prediction without an action plan has little value. The model is framed as a trigger for specific interventions, not as a label alone.
- **Applies:** reported model performance can differ from real-world impact, and humans must stay in the loop. The report shows that the per-order accuracy is modest and recommends against automated penalties for customers.
- **Applies:** poor-quality data leads to poor models. The audit of impossible values (Section 2.3) directly reflects this.
- **Applies:** models can perform differently in another setting. Performance on another retailer's data would need re-validation.

10.3 Dr. Siddharth K. Jabade — Universities as Innovation Infrastructure

- **Applies:** knowledge creation is not the same as innovation or impact, and a proof of concept is not a proof of value. This project is a proof of concept on a public, probably synthetic dataset; showing real value would require validation on a retailer's actual order data.
- **Applies:** his emphasis on starting from the **real industry problem** (his bus-conductor story about solving a problem that was not the real problem). The weak signal found here suggests that the true drivers of returns (e.g., size or fit issues, product-quality defects, stated return reasons) are not in these columns. Working with industry partners to obtain those variables is the most important next step.
- **Applies:** fail fast, fail cheap. Establishing early, with cheap baselines and honest confidence intervals, that the data has limited signal is itself a useful result.

10.4 Industry Research Panel — AI-Driven Research for a Sustainable and Innovative Future

- **Applies:** garbage in, garbage out — the panel noted that wrong data gives wrong results. Data validation was therefore the first step.

- **Applies:** formulate a problem in several ways and compare approaches. Five model families and two prediction scenarios were compared under the same protocol.
- **Applies:** validate AI outputs, and do not outsource judgment to AI. The rejected F1-optimal threshold (Section 7.4) is an example: an automatically optimised number that was checked and found meaningless.
- **Applies:** responsible use and governance — customer-level predictions require privacy safeguards and human oversight.

11. Conclusions and Limitations

11.1 Conclusions

> **OBSERVED RESULT (from the executed code)**
>
> The Kaggle dataset contains a genuine binary return target and can technically support the project; no target had to be constructed.
>
> The best model, XGBoost, reached holdout ROC-AUC 0.5957. It is better than chance and statistically better than Logistic Regression, but only slightly. The interpretable Logistic Regression model (0.5908) is a reasonable alternative when explainability matters more.
>
> The model is most useful for **ranking**: the highest-risk decile has a return rate of 63.1% vs a base rate of 47.5%.
>
> Product category, shipping method, device type, coupon use and payment method carry most of the (weak) signal.

11.2 Limitations

- **Weak signal.** All features have very small effect sizes, which caps achievable performance. The 0.5-threshold F1 of every model is below that of the trivial “always returned” rule.
- **Likely synthetic data.** 32.0% of rows contain impossible values, and some relationships run against business expectations (e.g., delivery delay). Findings may not transfer to real retail data.
- **No documentation.** Column meanings are inferred from names. Whether `past_return_rate` and `product_rating` are truly pre-outcome cannot be verified.
- **No timestamps or customer IDs.** Time-based validation (train on past, test on future) and customer-grouped validation are not possible, so real-world performance may be lower than the random-split estimates.
- **Unlabelled test set.** Predictions for test.csv were produced but cannot be scored locally.
- **Compute budget.** Hyper-parameter tuning used a 50,000-order subsample and small grids; a larger search is unlikely to change the conclusions given the weak signal.

11.3 Future work

- Obtain real order data with return reasons, product size/fit information, customer IDs and timestamps.
- Validate on future time periods and on unseen customers.
- Use a cost-sensitive threshold based on the actual cost of a return and of an intervention.
- Add text features from product descriptions and reviews, which often explain returns better than numeric order attributes.

Appendix A. Python Implementation

The complete pipeline is in `return_prediction_pipeline.py` (Python 3, scikit-learn, XGBoost, SHAP, pandas, SciPy, Matplotlib). One command reproduces every number and figure in this report:

```
pip install pandas numpy scipy scikit-learn xgboost shap matplotlib
python return_prediction_pipeline.py --data data --out results
```

Code component	KDD stage	Purpose
Profile block (read_csv, describe, value_counts)	2	File shapes, dtypes, missing values, duplicates, target balance, invalid values
Cleaner (custom transformer)	3	Sets impossible values to NaN inside the pipeline
FeatureEngineer (custom transformer)	4	Adds engineered features; removes delivery delay in the checkout scenario
ColumnTransformer: SimpleImputer + OneHotEncoder (+ StandardScaler)	3–4	Leakage-safe imputation, indicators, encoding, scaling
Statistical tests (scipy.stats)	5	Mann–Whitney U, point-biserial r, Chi-square, Cramér's V
GridSearchCV + cross_validate	6–7	Tuning and 5-fold stratified CV of every model
Holdout evaluation, threshold table, lift, bootstrap	7	Final metrics with confidence intervals
LR coefficients, permutation importance, SHAP	8	Explainability
results/results.json, figures/, submission.csv	—	All outputs used by this report; test-set predictions

Run time on the machine used: about 10 minutes (2 CPU cores). Test-set predictions: 50,000 orders, of which 37.5% are predicted as returns at threshold 0.5. These cannot be scored because test.csv has no labels.